

CAC Hub

Technische Dokumentation — Zentrales Dashboard fuer Pioneer CAC CD-Automatenwechsler im Netzwerk

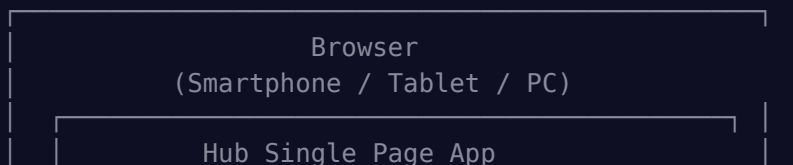
Inhalt

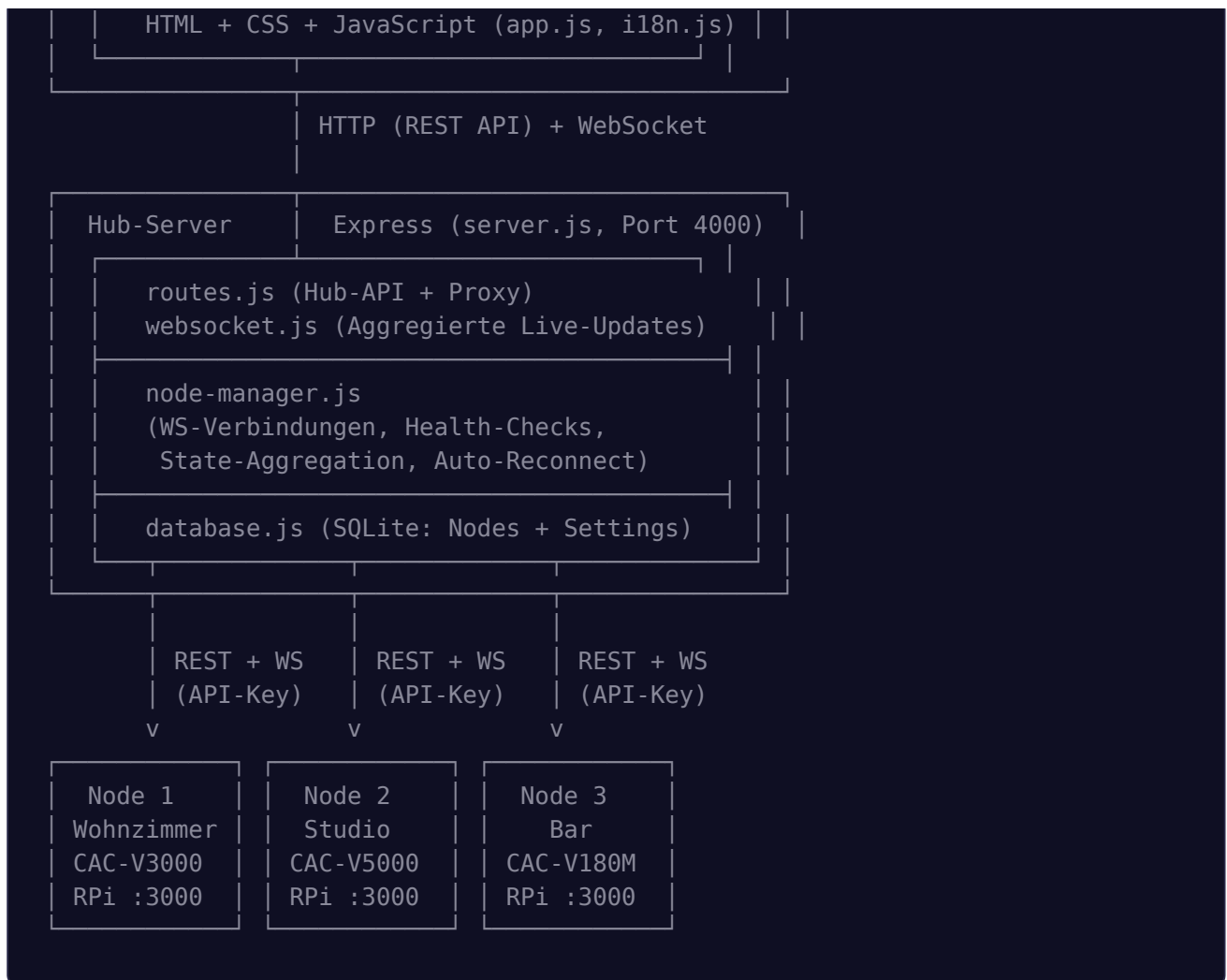
1. Konzept und Design-Prinzipien
2. Software-Architektur
3. Frontend
4. Datenfluss
5. Datenbank-Details
6. REST-API-Referenz
7. Deployment
8. Sicherheit
9. Bekannte Einschränkungen
10. Lizenz

Projektbeschreibung

Der CAC Hub ist ein zentrales Dashboard fuer die Verwaltung mehrerer Pioneer CAC CD-Automatenwechsler im Netzwerk. Er verbindet sich per REST-API und WebSocket mit beliebig vielen CAC Controller-Nodes und bietet eine einheitliche Oberflaeche fuer Monitoring und Steuerung aller Wechsler von einem einzigen Standort aus.

Systemarchitektur





1. Konzept und Design-Prinzipien

1.1 Dezentrale Architektur

Jeder CAC Controller Node laeuft als eigenstaendige Anwendung auf einem Raspberry Pi und steuert genau einen Pioneer CD-Automatenwechsler. Der Hub ist — jeder Node funktioniert vollstaendig ohne Hub.

1.2 Hub als Aggregator

Der Hub sammelt Zustandsinformationen von allen Nodes und leitet sie an die Hub-UI-Clients weiter. Er speichert keine CD-Daten oder Bibliotheken — diese verbleiben ausschliesslich auf den Nodes.

1.3 Vollstaendiger Fernzugriff

Ueber die Proxy-API hat der Hub auf alle Funktionen jedes Nodes: Player-Steuerung, Library-CRUD, Playlists, Scanner, MusicBrainz-Suche, Favoriten, Bewertungen, History, Einstellungen — alles.

1.4 API-Key-Authentifizierung

Jede Kommunikation zwischen Hub und Node ist per API-Key authentifiziert. Der Hub sendet den `X-API-Key` -Header bei jedem Proxy-Request und verbindet sich per `ws://node:port/?hub=1&apikey=<key>` fuer WebSocket.

2. Software-Architektur

2.1 Technologie-Stack

Komponente	Technologie	Version
Runtime	Node.js	≥ 18.x
Web-Framework	Express	5.x
WebSocket	ws	8.x
Datenbank	better-sqlite3	11.x
Frontend	Vanilla JS (SPA)	—
i18n	Eigene Loesung	DE/EN

2.2 Projektstruktur

```
cac-controller-hub/
├─ server.js           # Einstiegspunkt (Port 4000)
├─ package.json
├─ cac-hub.service     # Systemd-Unit
├─ src/
│   ├─ database.js     # SQLite (Nodes, Settings)
│   ├─ node-manager.js # WebSocket-Verbindungen zu Nodes
│   ├─ routes.js       # Hub-API + Proxy-Routen
│   └─ websocket.js    # WebSocket fuer Hub-UI
└─ public/
```

```

|   |— index.html          # Hub SPA
|   |— css/app.css        # Dark Theme
|   |— js/
|       |— app.js          # Frontend-Logik
|       |— i18n.js         # Uebersetzungen (DE/EN)
|— data/
|   |— cac-hub.db          # SQLite-Datenbank

```

2.3 Modul-Uebersicht

server.js — Einstiegspunkt

Initialisiert Datenbank, NodeManager, Express-Server und WebSocket. Verbindet sich beim Start mit allen aktivierten Nodes. Graceful Shutdown per SIGINT/SIGTERM.

database.js — SQLite-Datenbank

Speichert die Node-Konfiguration und Hub-Einstellungen:

```

nodes (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  name TEXT,                -- Anzeigename
  url TEXT NOT NULL,        -- z.B. http://192.168.1.100:3000
  api_key TEXT,             -- API-Key fuer Authentifizierung
  room TEXT,               -- Raum/Standort
  model TEXT,              -- Pioneer-Modell (automatisch)
  enabled INTEGER DEFAULT 1, -- Aktiviert/Deaktiviert
  sort_order INTEGER DEFAULT 0,
  created_at TEXT,
  updated_at TEXT
)

settings (
  key TEXT PRIMARY KEY,
  value TEXT
)

```

Standard-Einstellungen: `hub_port`, `hub_name`, `language`, `reconnect_interval`, `health_check_interval`.

node-manager.js — Node-Verbindungsmanager

Zentrales Modul fuer die Kommunikation mit allen Nodes:

- Verbindet sich per `ws://node:port/?hub=1&apikey=<key>` mit jedem aktivierten Node
 - Empfängt `hubInit` -Nachricht bei Verbindung (Name, Raum, Modell, Player-Status)
 - Empfängt alle Broadcast-Events (playerState, scanProgress, playModeChange, etc.)
 - Aggregiert den Zustand aller Nodes in einer zentralen Map
-
- Bei Verbindungsverlust wird alle 10 Sekunden (konfigurierbar) ein Wiederverbindungsversuch gestartet
 - Node-Daten werden vor dem Reconnect aus der DB aktualisiert (falls URL/Key geändert)
-
- Alle 30 Sekunden (konfigurierbar) wird per REST-API der Status jedes Nodes abgefragt
 - Dient als Backup fuer den Fall, dass WebSocket-Events verloren gehen
-
- `proxyRequest(nodeId, method, path, body)` — leitet beliebige API-Aufrufe an den Node weiter
 - `proxyBinary(nodeId, path)` — leitet binaere Inhalte (Cover-Bilder) weiter
 - Fuegt automatisch den `X-API-Key` -Header hinzu
-
- `nodeOnline` — Node ist verbunden
 - `nodeOffline` — Verbindung verloren
 - `nodeState` — Vollstaendiger Zustand empfangen (hubInit)
 - `nodeEvent` — Einzelnes Event von einem Node

routes.js — Hub-API und Proxy

- `GET /api/hub/status` — Hub-Uebersicht mit allen Nodes
- CRUD fuer Nodes: `GET/POST/PUT/DELETE /api/nodes`
- `POST /api/nodes/test` — Verbindungstest zu einem Node

- `GET/PUT /api/settings` — Hub-Einstellungen
- `ALL /api/nodes/:id/proxy/*path` — Leitet jeden API-Aufruf an den Node weiter
- `GET /api/nodes/:id/cover/*path` — Leitet Cover-Bilder weiter

Der Proxy ist transparent: Der Hub-Client ruft z.B. `/api/nodes/1/proxy/library` auf, der Hub leitet dies als `GET http://node:3000/api/library` mit API-Key weiter.

websocket.js — Hub-WebSocket

Verwaltet WebSocket-Verbindungen von Hub-UI-Clients:

- Bei Verbindung: Sendet `init` -Nachricht mit dem aktuellen Zustand aller Nodes
- Leitet alle Events vom NodeManager an alle Hub-Clients weiter
- Jede Nachricht enthaelt die `nodeId` , damit der Client weiss, welcher Node betroffen ist

3. Frontend

3.1 Dashboard

Das Dashboard zeigt alle konfigurierten Nodes als Karten in einem responsiven Grid:

- `status` : Gruener Punkt = online, roter Punkt = offline
- `player` : Fuer jeden Node werden beide Player mit Modus, Disc und Track angezeigt
- `controls` : Play, Pause, Stop und Next direkt auf der Dashboard-Karte
- `detail` : Oeffnet die Detail-Ansicht des Nodes

3.2 Node-Detail-Ansicht

Vollstaendige Steuerung eines ausgewaehlten Nodes mit vier Sub-Tabs:

- Beide Player nebeneinander (responsiv: untereinander auf Mobilgeraeten)

- Cover-Bild, Disc-Titel, Kuenstler, Slot-Nummer
 - Track-Nummer und Track-Titel
 - Zeitanzeige (Echtzeit via WebSocket)
 - Transport-Controls: Previous, Stop, Play, Pause, Next
 - Volume-Slider
 - Track-Liste der aktuellen CD (klickbar zum Abspielen)
 - Play-Modi: Continuous, Gapless, Shuffle
-
- Durchsuchbare Liste aller CDs des Nodes
 - Cover-Bilder (via Proxy geladen)
 - Detail-Modal mit Track-Liste
 - Load-Buttons fuer Player 1 und Player 2
-
- Liste aller Playlists des Nodes
 - Play-Button zum Starten
-
- Start/End-Slot eingeben
 - Scan starten / abbrechen
 - Live-Fortschrittsbalken via WebSocket

3.3 Einstellungen

- `nodes` : Liste aller Nodes mit Status, Loeschen-Button
- `node` : URL, API-Key, Name, Raum eingeben; Verbindungstest; Auto-Fill von Name/Raum/Modell
- `node` : Name, Port, Sprache

3.4 Internationalisierung

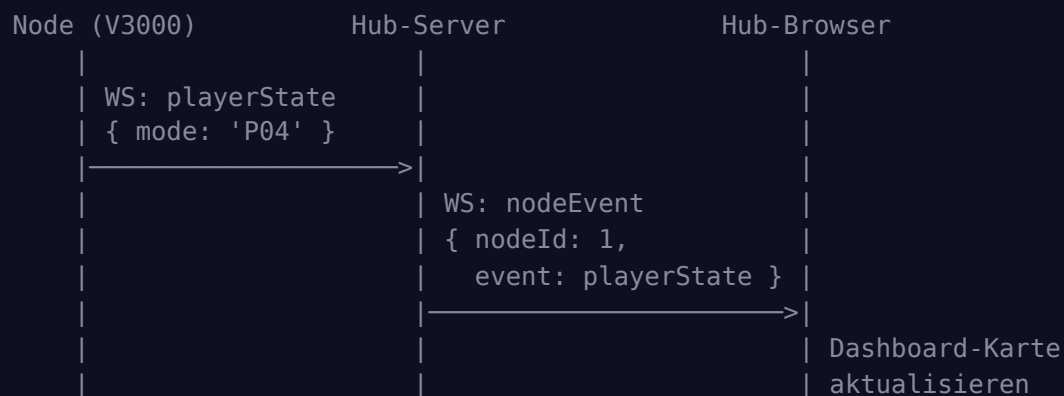
Wie beim Node: `data-i18n` -Attribute, `t('key')` -Funktion, Sprachumschaltung per Button. Ca. 80 Uebersetzungsschluesel fuer Deutsch und Englisch.

4. Datenfluss

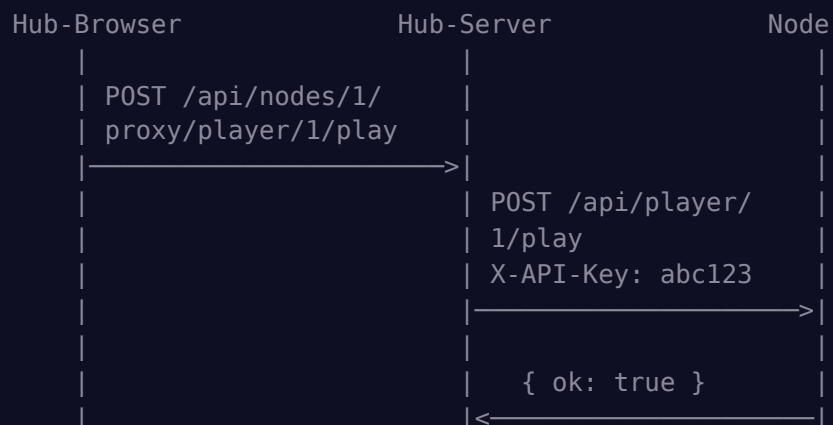
4.1 Hub-Start

1. Datenbank initialisieren
2. NodeManager erstellen
3. Express-Server starten (Port 4000)
4. Hub-WebSocket starten
5. Fuer jeden aktivierten Node:
 - a. WebSocket-Verbindung oeffnen
 - b. hubInit empfangen -> Zustand speichern
 - c. Health-Check-Timer starten

4.2 Dashboard-Update bei Player-Aenderung



4.3 Proxy-Kommando (z.B. Play druecken)




```
{ ok: true }
```

4.4 Library laden



5. Datenbank-Details

5.1 Schema

Die Hub-Datenbank ist minimalistisch — sie speichert nur Node-Konfigurationen und Hub-Einstellungen. Alle CD-Daten, Playlists, Favoriten etc. verbleiben auf den Nodes.

5.2 WAL-Modus

Write-Ahead Logging fuer bessere Performance bei gleichzeitigem Lesen und Schreiben.

5.3 Standard-Einstellungen

Schluessel	Standard	Beschreibung
hub_port	4000	HTTP-Port

hub_name	CAC Hub	Anzeigename
language	auto	Sprache (auto/de/en)
reconnect_interval	10000	Reconnect-Intervall (ms)
health_check_interval	30000	Health-Check-Intervall (ms)

6. REST-API-Referenz

6.1 Hub-Endpunkte

Methode	Endpunkt	Beschreibung
GET	/api/hub/status	Hub-Uebersicht mit allen Nodes und Status
GET	/api/nodes	Alle Nodes mit Verbindungsstatus
POST	/api/nodes	Node hinzufuegen { url, api_key, name, room }
GET	/api/nodes/:id	Einzelnen Node mit Status abrufen
PUT	/api/nodes/:id	Node aktualisieren
DELETE	/api/nodes/:id	Node entfernen
POST	/api/nodes/test	Verbindungstest { url, api_key }
GET	/api/settings	Hub-Einstellungen abrufen
PUT	/api/settings	Hub-Einstellungen aktualisieren

6.2 Proxy-Endpunkte

Methode	Endpunkt	Beschreibung
ALL	/api/nodes/:id/proxy/*	Beliebigen API-Aufruf an Node weiterleiten
GET	/api/nodes/:id/cover/*	Cover-Bild von Node laden

Beispiele:

- GET /api/nodes/1/proxy/library → GET http://node:3000/api/library
- POST /api/nodes/1/proxy/player/1/play → POST http://node:3000/api/player/1/play

- `PUT /api/nodes/1/proxy/library/42` → `PUT http://node:3000/api/library/42`
- `GET /api/nodes/1/cover/cover_42.jpg` → `GET http://node:3000/covers/cover_42.jpg`

6.3 WebSocket-Nachrichten

Typ	Richtung	Beschreibung
<code>init</code>	Hub → Client	Initialer Zustand aller Nodes
<code>nodeOnline</code>	Hub → Client	Node verbunden
<code>nodeOffline</code>	Hub → Client	Node getrennt
<code>nodeState</code>	Hub → Client	Vollstaendiger Zustand eines Nodes
<code>nodeEvent</code>	Hub → Client	Weitergeleitetes Event (playerState, scanProgress, etc.)

7. Deployment

7.1 Systemd-Service

```
[Unit]
Description=CAC Hub - Central Dashboard for Pioneer CD Autochanger Nodes
After=network.target

[Service]
Type=simple
User=pi
WorkingDirectory=/opt/cac-controller-hub
ExecStart=/usr/bin/node server.js
Restart=always
RestartSec=5
Environment=NODE_ENV=production

[Install]
WantedBy=multi-user.target
```

7.2 Hub und Node auf demselben Pi

Beide Anwendungen koennen gleichzeitig auf demselben Raspberry Pi laufen:

- Node auf Port 3000 (Standard)
- Hub auf Port 4000 (Standard)
- Node im Hub hinzufuegen mit URL `http://localhost:3000`

7.3 Netzwerk-Anforderungen

- Alle Nodes muessen vom Hub-Pi aus per HTTP erreichbar sein
 - Standard-Ports: Node 3000, Hub 4000
 - Empfohlen: Statische IP-Adressen oder DNS fuer alle Pis
-

8. Sicherheit

8.1 API-Key-Authentifizierung

- Jeder Node generiert einen 32-stelligen alphanumerischen API-Key
- Der Hub speichert den Key verschluesselt in seiner SQLite-Datenbank
- Alle Proxy-Requests enthalten den `X-API-Key`-Header
- WebSocket-Verbindungen erfordern den Key als Query-Parameter

8.2 Empfehlungen

- Nur im lokalen Netzwerk betreiben
 - Kein Port-Forwarding zum Internet
 - Starke, zufaellige API-Keys verwenden (die Node-UI generiert automatisch sichere Keys)
 - Bei Bedarf: Nginx als Reverse-Proxy mit TLS
-

9. Bekannte Einschraenkungen

1. Der Hub speichert keine CD-Daten. Bei Verlust der Verbindung zu einem Node sind dessen Daten im Hub nicht mehr sichtbar.
 2. Der Hub kann nicht Audio von einem Node auf einem anderen Geraet wiedergeben. Jeder Node hat seinen eigenen analogen Audioausgang.
 3. Playlists koennen nur CDs eines einzelnen Nodes enthalten. Eine uebergreifende Playlist ueber mehrere Wechsler ist nicht unterstuetzt.
 4. Die Kommunikation zwischen Hub und Nodes erfolgt unverschluesstelt (nur fuer LAN gedacht).
-

10. Lizenz

MIT License — Copyright © 2025 Dirk Jensen — siehe LICENSE.
